Review paper

Phases of compiler design

The six-phase model, measured against Zero — a compiler that reports its own phase structure as machine-readable data.

Submitted by

Harshit Khemani

Co-authors

Kush Ahuja, Mohit Kumar Mishra, Kushagra Agrawal

Submitted to

Ms. Ankita Sharma

Read online

zero.khe.money

The web edition carries the interactive phase explorer and the full dataset.

Abstract

Compiler construction is conventionally taught as a pipeline of six phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and target code generation, with symbol-table management and error handling running alongside all six. This decomposition is a teaching abstraction. Production compilers rarely expose it, so students seldom see the phases as separable, measurable objects.

This paper reviews that model against Zero (https://zerolang.ai/) 0.3.4, an experimental graph-first systems language from Vercel Labs whose compiler reports its own phase names, per-phase timings, symbol tables, and diagnostics as structured JSON. We construct a corpus of 8 Zero programs (680 lines, 3,411 tokens, 2,216 graph nodes) and a second corpus of 10 deliberately malformed programs, then measure the compiler across 64 program-target build combinations.

We report three findings. First, the phases do not disappear in a graph-first design — they *relocate*. Lexical, syntactic, and semantic analysis move to an ingestion boundary that admits programs into a

persistent graph, after which the compile path performs only lowering, code generation, and linking. All 7 front-end error cases were refused at that boundary, and in none of them did the malformed program enter the graph store. Second, lowering accounts for effectively all measurable phase time (100% of reported phase milliseconds); the entire front end runs below the compiler's 1 ms reporting resolution because it reads stored facts instead of recomputing them. Third, the front end and the back end accept *different languages*: 17 of 64 builds failed with BLD004 on programs that had already passed `zero check`. The compiler detects this — it records `buildable: false` and names the offending construct — but reports `ok: true` with an empty top-level `diagnostics` array in the very same document.

We argue that Zero's value to compiler pedagogy lies less in its agent-oriented framing than in its observability: it is the first widely available compiler we know of in which a student can print the phase structure, time each phase, inspect the symbol table as data, and watch a program be rejected at three distinct gates.

8	2,216	47/64	100%	10
Programs in corpus	Graph nodes analysed	Builds that succeeded	Phase time in lowering	Error cases

All figures produced by `tools/capture.mjs` against Zero 0.3.4 (build 5b3a90a) on 13th Gen Intel(R) Core(TM) i5-13450HX, 16 cores, win32/x64. Captured 2026-08-09.

1. Introduction

Every undergraduate compilers course opens with the same diagram: source text enters at the top, passes through a vertical stack of labelled boxes, and machine code emerges at the bottom. Two narrow rectangles run down the side of the stack, touching every box — the symbol table and the error handler. The diagram is due in its modern form to Aho, Lam, Sethi and Ullman (https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472), and it has organised the field for four decades.

The diagram is also, in an important sense, unfalsifiable by students. Real compilers fuse phases for speed, interleave them for incrementality, and expose none of the boundaries. A student who runs `gcc` or `rustc` sees a binary appear and, on failure, a paragraph of English. The phases are real, but they are not *visible*.

This paper asks a narrow question: **does the classical six-phase model still describe a compiler built on a fundamentally different substrate, and if not, how does it differ?** We answer it empirically — by building a corpus, instrumenting the compiler through its own reporting interfaces, and measuring — rather than by reading documentation.

Section 2 introduces Zero: what it is, how it departs from mainstream languages, and the argument its designers make for why it is needed. Section 3 sets out the classical phase model as the baseline. Section 4 places the two side by side. Sections 5 and 6 describe our method and report what we measured. Sections 8 to 10 discuss what follows, where our evidence is weak, and how this relates to prior work.

Our contributions are: (i) a phase-by-phase mapping between the classical model and Zero's reported pipeline, with the command that makes each phase observable; (ii) a quantitative account of where compile time actually goes; (iii) an error corpus that localises each diagnostic to the gate that emits it, demonstrating three distinct admission stages rather than one; (iv) a 8 × 8 backend coverage matrix documenting a front-end/back-end acceptance gap; and (v) an interactive explorer that steps one program through all six phases using the compiler's real output.

---

2. What Zero is

Zero (https://zerolang.ai/) is an experimental systems programming language released by Vercel Labs in May 2026 under Apache-2.0, with source on GitHub (https://github.com/vercel-labs/zerolang). It compiles to standalone native executables, has no garbage collector, gives explicit control over memory, and sits in roughly the design space of C and Zig. Its compiler core is written in C. Programs use the `.0` extension. We study version 0.3.4, build 5b3a90a.

Its distinguishing premise is stated in its own tagline: *the programming language for agents*. Zero is built on the assumption that AI agents, rather than humans, will be the primary consumers of compiler output — and the design follows that assumption further than any other language we are aware of.

p01_hello / src/main.0 — the smallest complete program

```
pub fn main(world: World) -> Void raises {
    check world.out.write("hello from zero\n")
}
```

Figure 1. `pub fn` exports. `World` carries runtime capabilities. `raises` marks the function fallible, and `check` propagates failure from a fallible call. This program compiles to a 1536-byte native executable in 4 ms of lowering.

2.1 How it differs from current languages

Four departures matter for this paper.

The semantic graph is the program, not the text. This is the largest departure. A Zero package compiles from a binary `zero.graph` store holding declarations, types, calls, scopes, imports, capabilities and source-map facts as rows in sixteen relations. The `.0` files a human reads are a *projection* of that graph. In every other mainstream language, text is authoritative and the compiler's internal representation is derived and discarded; Zero inverts that relationship (https://zerolang.ai/concepts/semantic-vs-text). Agents edit through `zero patch` against stable node handles rather than line ranges.

Diagnostics are data, not prose. Every command accepts `--json` against a versioned schema. Each diagnostic carries a stable code (`NAM003` , `TYP009` , `BLD004`), a span, structured `expected` / `actual` facts, a `fixSafety` rating, and often a typed `repair` identifier. The compiler does not merely describe a problem; it names a repair operation and rates whether a machine may apply it unsupervised.

Effects and capabilities are explicit and target-checked. A function performing I/O receives a `World` handle and is marked `raises`; callers must use `check` or `rescue`. Each compilation target independently declares which capabilities it provides, so a program using the network is rejected at compile time for a target that declares no `net` capability — a property we exercise directly in §6.4.

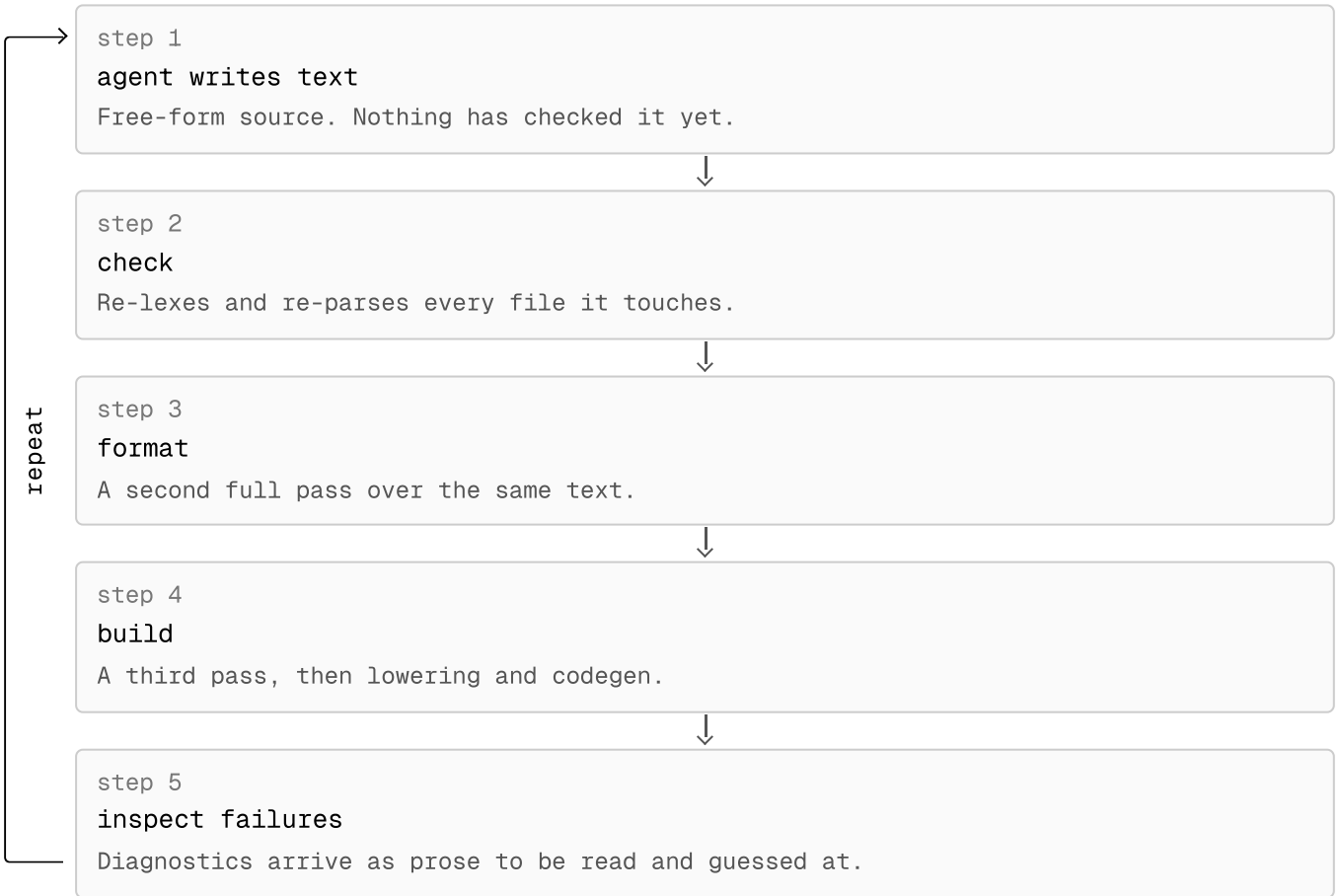
The toolchain reports itself. A single `zero check --json` returns the phase list with timings, the graph table row counts, the resolved call graph with contracts, cache keys with hit status, interface fingerprints per module, and a target readiness report. There is also `zero tokens`, which counts a program in LLM tokens — a command that only makes sense if you believe context-window cost is an engineering constraint.

2.2 Why its designers argue it is needed

The case Zero makes is about the cost of a translation layer. In a conventional agent coding loop, an agent writes text, the compiler renders its objection as English, and the agent parses that English back into an intent it already had. Zero's graph architecture documentation (https://zerolang.ai/concepts/graph-architecture) frames the contrast as two loops.

Traditional agent source loop

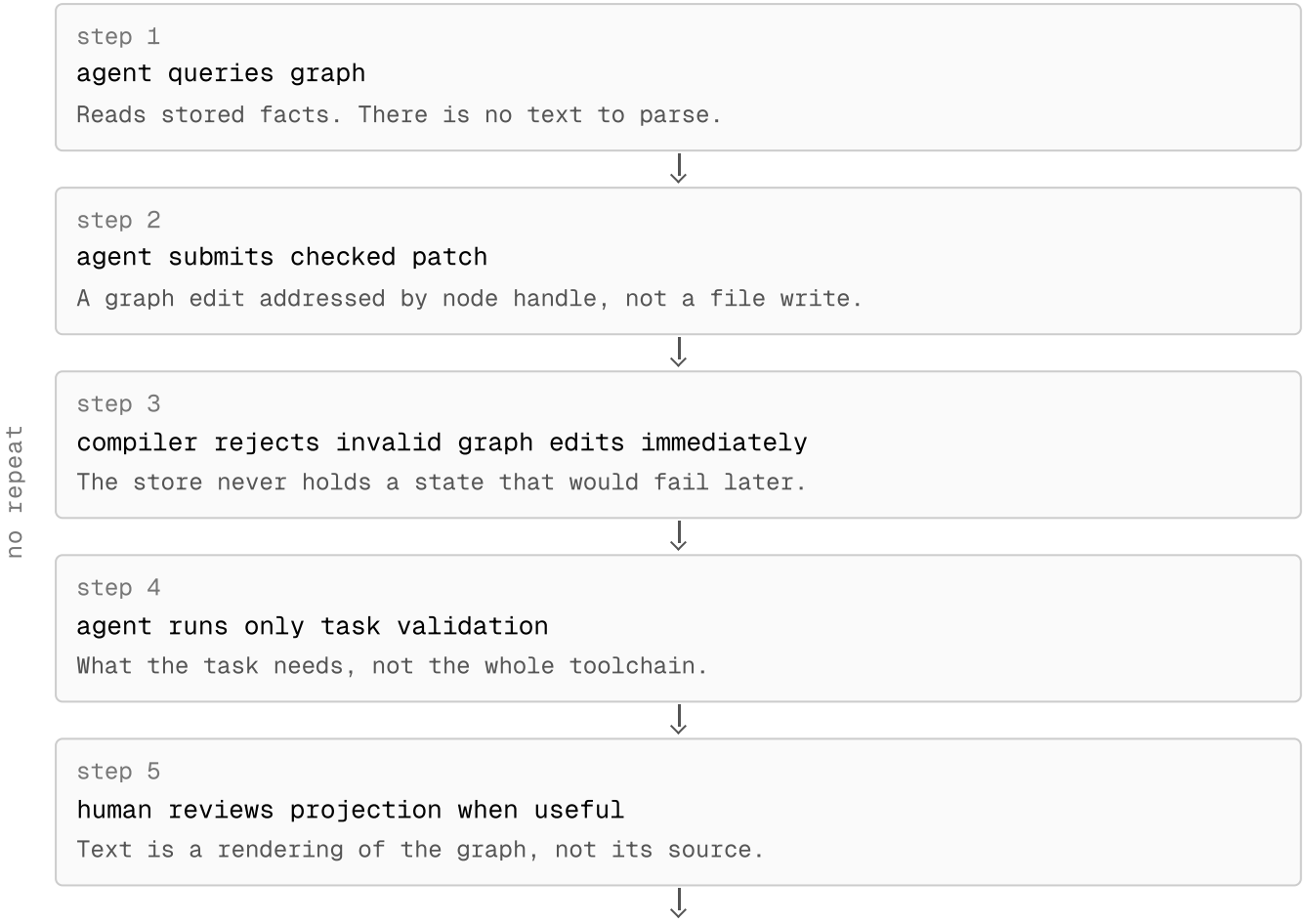
a cycle · 5 steps · repeats until the build passes



After step 5 the loop returns to step 1: `(repeat)`. Every pass re-derives the same tokens, the same tree and the same bindings from text that has already been read, and the agent only learns an edit was invalid at the end of the pass.

Zero graph loop

largely linear · 5 steps · terminates



There is no (`repeat`) edge. An invalid edit is refused at step 3, at submission time, rather than discovered at the end of a build; the store therefore never holds a state the next pass would have to find and undo.

Three specific failures are claimed. *Line ranges are the wrong handle* — they drift the moment anything reformats, whereas semantic node identifiers do not. *English is a lossy encoding of compiler state* — the compiler knew the exact repair and discarded that structure to print a sentence. *Feedback latency bounds the loop* — millisecond builds change what an agent can afford to attempt.

A fourth motive is arguably the strongest and is less often stated: **capability containment**. For code an agent wrote and no human read line by line, a compiler that refuses to build a program using a capability the target does not declare is a real safety property, not a convenience.

We record the counterarguments in the same breath, because they are substantial. Structured diagnostics are not new: rustc (https://doc.rust-lang.org/rustc/json.html) and TypeScript have emitted machine-readable errors for years, so Zero's claim must rest on completeness rather than novelty. And the adoption argument cuts hard against it — models write best in the languages that dominate their training data, so a language with a few thousand GitHub stars is one that agents are, today, measurably worse at than Go or Rust, however good its interfaces are. Zero is betting that a tight verifiable loop outruns familiarity. That bet is unproven, and this paper does not attempt to settle it.

3. How traditional compilers work

The canonical decomposition treats compilation as a sequence of meaning-preserving translations between representations. Each phase consumes one representation and produces the next.

The **analysis** half — phases one to three, the front end — determines what the program means. Lexical analysis groups characters into tokens, discarding whitespace and comments. Syntax analysis arranges tokens into a tree reflecting the grammar. Semantic analysis annotates that tree with types, resolves every name to a declaration, and rejects programs that are well-formed but meaningless.

The **synthesis** half — phases four to six, the back end — determines how the program runs. Intermediate code generation produces a representation independent of both source language and target machine. Optimization rewrites it to be cheaper without changing behaviour. Target code generation selects instructions, allocates registers, and emits an object.

Two activities refuse to sit in any single box. The **symbol table** is written by the front end and read by every later phase. **Error detection** occurs in all six, and the phase that detects an error largely determines how good the message can be: a lexer knows only about characters, a type checker knows about declarations.

That last point is the one we test. If phases are genuinely separable, a program should be rejectable at a specific phase, and errors should partition cleanly by the phase with enough information to detect them. Section 6.4 puts it to the experiment.

Phases 1–3 · at the ingestion gate

`zero import · once per edit`

`phase 1`

Lexical analysis

`Character stream → Token stream`

`parse` `locus: ingestion`

↓

`phase 2`

Syntax analysis

`Token stream → Parse tree / AST`

`parse` `locus: ingestion`

↓

`phase 3`

Semantic analysis

`AST + symbol table → Annotated AST, type facts`

`resolve` `interface` `check` `locus: both`

`Runs at both: admitted at ingestion, re-consulted from stored facts on the compile path.`

These three run when text enters the graph and are not re-run to produce a build. Phase 3 is the boundary case: it is admitted here and then re-consulted on the compile path from stored symbol, type and scope tables rather than rebuilt.

The admission boundary

above: once per edit · below: once per build

Phases 4–6 · on the compile path

zero build · once per build

phase 4

Intermediate code generation

Annotated AST → Intermediate representation

lower | locus: compile-path

phase 5

Code optimization

Intermediate representation → Improved IR

lower | codegen | locus: compile-path

phase 6

Target code generation

Optimized IR → Machine code / object

codegen | object | link | locus: compile-path

These three are what a build actually costs. They read a graph whose names are already bound and whose types are already recorded, which is why the compile path can begin at lowering.

cross-cutting · spans phases 1-6

Symbol table management

Classical

A data structure rebuilt by the front end on every compilation.

As Zero realises it

A persisted set of graph tables (schema, package, module, declaration, scope, import, symbol, type, effect, capability, ownership, resource, node, edge, projection, sourceMap) carried between runs in zero.graph and addressable by stable node handles.

probe: zero query --json --full

cross-cutting · spans phases 1-6

Error detection and reporting

Classical

Diagnostics rendered as prose for a human reader.

As Zero realises it

Structured records with a stable code, a span, expected/actual facts, a fixSafety rating and an optional typed repair id — designed to be consumed by a program, not parsed from English.

probe: zero check --json | zero explain --json | zero fix --plan --json

The classical six-phase stack, drawn once. The split is not a redrawing of the phases but a statement about cadence: phases 1–3 are paid when an edit is admitted, phases 4–6 when an artifact is requested. The two panels are the cross-cutting concerns the textbook draws as vertical bars beside the stack; in Zero both are persisted

and directly inspectable, which is why the split above is observable rather than asserted. Phase numbering, input/output pairs, Zero phase names and locus values are read from `lib/phases.ts`. On a narrow screen the two bars move underneath the stack rather than beside it.

---

4. How Zero's compiler differs

Zero's own compile-path documentation (https://zerolang.ai/concepts/compile-path) states the contrast directly. A conventional compiler runs source files through a lexer and parser to an AST, then name resolution, type checking, IR lowering, optimization and codegen. Zero starts from the graph store and runs repository graph tables through semantic validation, type checking, MIR and backend facts, to direct codegen.

Traditional parse-first path

as documented for Rust, Go, Zig, C · 9 stages

T1
source files

↓

T2
lexer/parser

↓

T3
AST

↓

T4
name resolution

↓

–
No counterpart. The parse-first path has no persisted store to read.

↓

–
No counterpart. Nothing validates an edit before the front end runs.

↓

T5
type checking

↓

T6
IR lowering

↓

Zero graph-first path

Zero 0.3.4 package compilation · 7 stages

Z1
zero.graph

↓

moved off this path

Lexing, parsing and name resolution run once, at the ingestion gate, when text enters the graph. Package compilation reads a binary store, so none of the three stages at left runs here.

probe: zero import · zero parse --json

↓

Z2
repository graph tables
Read, not rebuilt.

↓

Z3
semantic validation
Rejects an invalid graph edit.

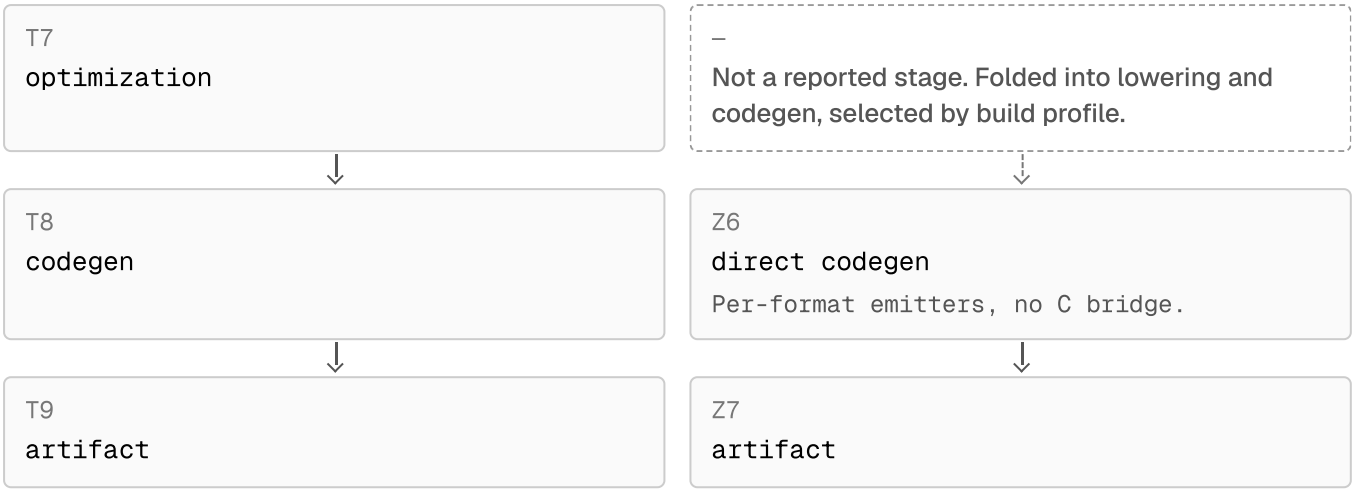
↓

Z4
type checking
Over stored type facts.

↓

Z5
MIR and backend facts
Contracts verified before emission.

↓



Both tracks read top to bottom. Slots are aligned, so a dashed box marks a stage one path has and the other does not: three traditional stages leave the Zero build path entirely, one more (optimization) is folded into two others, and two Zero stages have no traditional counterpart. Stage labels are the documented names, unabbreviated.

Text alternative to the diagram above: the two compile paths as ordered stage sequences, 9 stages against 7. Rows are positional only — row n holds stage n of each sequence and does not assert a correspondence between them. — means the sequence has ended. No units; this table is definitional.

#	Traditional parse-first stage	Zero graph-first stage
1	source files	zero.graph
2	lexer/parser	repository graph tables
3	AST	semantic validation
4	name resolution	type checking
5	type checking	MIR and backend facts
6	IR lowering	direct codegen
7	optimization	artifact
8	codegen	–
9	artifact	–

The consequential difference is what is *missing* from the second sequence. There is no lexer, no parser, and no separate name-resolution stage on the compile path, because by the time a program is in the graph store its names are already bound and its types already recorded. Those stages still exist — they simply run somewhere else, at the boundary where text is admitted into the graph.

This is why the compiler reports `resolve` *before* `parse` in its own phase list, an ordering that reads as a typo until you understand the substrate. Resolution against stored symbol facts is a load, and a load can precede parsing.

The eight phases Zero reports, in the order it lists them, are:

- `resolve` — Binds names against stored symbol facts.
- `parse` — Graph-native; no character scan on the package path.
- `interface` — Public symbol surface and import graph fingerprinting.
- `check` — Types, effects, ownership, capability requirements.
- `lower` — Graph HIR to MIR, with contract verification before emission.
- `codegen` — MIR to target machine code via direct emitters.
- `object` — Object-file construction in the target format.
- `link` — The only phase the compiler marks non-cacheable.

And the two cross-cutting concerns the textbook draws as vertical bars are, in Zero, first-class inspectable objects:

Symbol table management. A persisted set of graph tables (schema, package, module, declaration, scope, import, symbol, type, effect, capability, ownership, resource, node, edge, projection, sourceMap) carried between runs in zero.graph and addressable by stable node handles. Probe: `zero query --json --full`

Error detection and reporting. Structured records with a stable code, a span, expected/actual facts, a fixSafety rating and an optional typed repair id — designed to be consumed by a program, not parsed from English. Probe: `zero check --json | zero explain --json | zero fix --plan --json`

---

5. Methodology

5.1 Corpus construction

We wrote 8 Zero packages of increasing size and feature coverage, from a six-line hello-world to a 304-line arithmetic tokenizer. Each was developed until `zero check`, `zero test` and `zero run` all succeeded, or until the obstruction was documented as a finding. The corpus totals 680 non-empty lines, 3,411 tokens and 23 passing test blocks. Feature coverage was chosen to exercise distinct compiler tables: control flow, shapes and enumerations, fallible functions and effects, borrowing and spans, and generics.

5.2 Error corpus

Measuring where errors are detected requires programs that fail on purpose. We wrote 10 minimal packages, each violating exactly one rule and targeting a specific phase: an unclassifiable byte, an unclosed block, a call to an undeclared function, a type mismatch, a write through an immutable binding, an unchecked fallible call, a stack frame over budget, a capability the selected target does not provide, a `choice`-typed parameter, and a `match` statement. Each carries a `case.json` declaring the phase it targets *before* measurement, so the mapping in §6.4 is a prediction rather than a post-hoc fit.

5.3 Instrumentation

One harness, `tools/capture.mjs`, drives every measurement and writes a single JSON document. Per program it captures the token stream (`zero tokens`), the parse summary (`zero parse`), the full semantic report (`zero check --json`), the graph (`zero query --full`), source mappings, phase timings (`zero time --json`), and artifact measurements (`zero size`, `zero mem`). It then builds every program for every target the compiler advertises.

Two measurement decisions deserve statement. We report phase timings from `zero time` rather than `zero check`, because the former drives the pipeline through emission and so attributes lowering cost. And we report wall-clock separately from in-process phase time, as median of five runs, because process startup (~40 ms) dominates at this program scale and would otherwise swamp the phase signal.

5.4 Reproducibility

Every number in this paper derives from one machine-generated dataset. No figure is transcribed by hand; the prose reads the same JSON the charts do, so text and evidence cannot drift apart. Appendix A gives the commands to regenerate it. The environment was an 13th Gen Intel(R) Core(TM) i5-13450HX with 16 logical cores and 31.7 GB of memory, running win32/x64.

---

6. Results

6.1 Mapping the classical phases onto Zero

Zero reports eight phases; the classical model names six. They do not correspond one-to-one, and the mismatches are informative.

The six canonical front-to-back compiler phases (Aho, Lam, Sethi and Ullman) against the phase names Zero 0.3.4 row stating where the realisation departs from the textbook. Counts and units: none — this table is definitional.

#	Classical phase	Input → output	Zero phases
1	Lexical analysis	Character stream → Token stream	`parse`
	Divergence — Runs only when text enters the system. Package compilation reads a binary graph store and neve		
2	Syntax analysis	Token stream → Parse tree / AST	`parse`
	Divergence — The tree is not the compiler's working representation. Parsing exists to admit text into the graph;		
3	Semantic analysis	AST + symbol table → Annotated AST, type facts	`resolve` `interface` `check`
	Divergence — Split across three reported phases and persisted as typed graph facts. The symbol table is not rel		
4	Intermediate code generation	Annotated AST → Intermediate representation	`lower`
	Divergence — Lowering runs graph HIR to MIR directly, and MIR contracts are verified before emission. The IR is direct backend has no LLVM path. The only observable is its size in bytes.		
5	Code optimization	Intermediate representation → Improved IR	`lower` `codegen`
	Divergence — Not a separately reported phase. Optimization is selected by build profile rather than exposed as		
6	Target code generation	Optimized IR → Machine code / object	`codegen` `object` `link`
	Divergence — Three reported phases, not one. Direct per-format emitters replace a C bridge, and only `link` is ma		

Semantic analysis fragments into three reported phases — `resolve`, `interface`, `check` — because interface fingerprinting is separated out to drive incremental invalidation. Optimization has *no* reported phase at all: it is selected by build profile, and its cost folds into `lower` and `codegen`. And the IR is never printed — `--emit llvm-ir` fails with BLD004 on every target because the direct backend has no LLVM path, leaving the lowered module's *size* as the only observable.

6.2 Where compile time actually goes

The result is stark enough to state plainly: across all 8 programs, 100% of reported phase milliseconds are spent in `lower`. Every other phase — resolve, parse, interface, check, codegen, object, link — reports

0 ms.

Where the milliseconds go

Pick a program, smallest to largest. The left panel is every phase time the compiler reported for it; the right panel is the size of the IR that lowering produced, placed against the whole corpus.

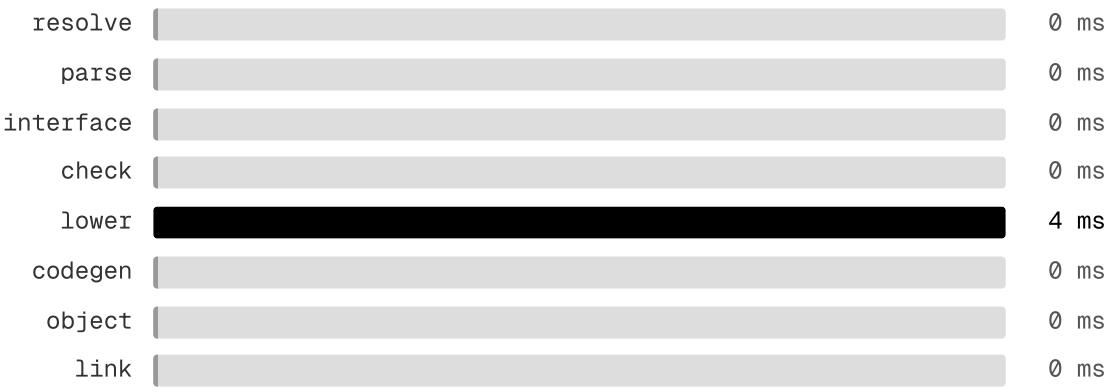
Program, by non-empty source lines

p01_hello 6 lines | p02_arith 31 lines | p07_generics 45 lines | p03_control 54 lines | p05_errors 54 lines | p04_shapes 85 lines | p06_memory 101 lines | p08_lexer 304 lines

6	4 ms	4 ms	2,185 B
Non-empty source lines	Total reported phase time	Of which lower	Lowered IR bytes

Reported phase time

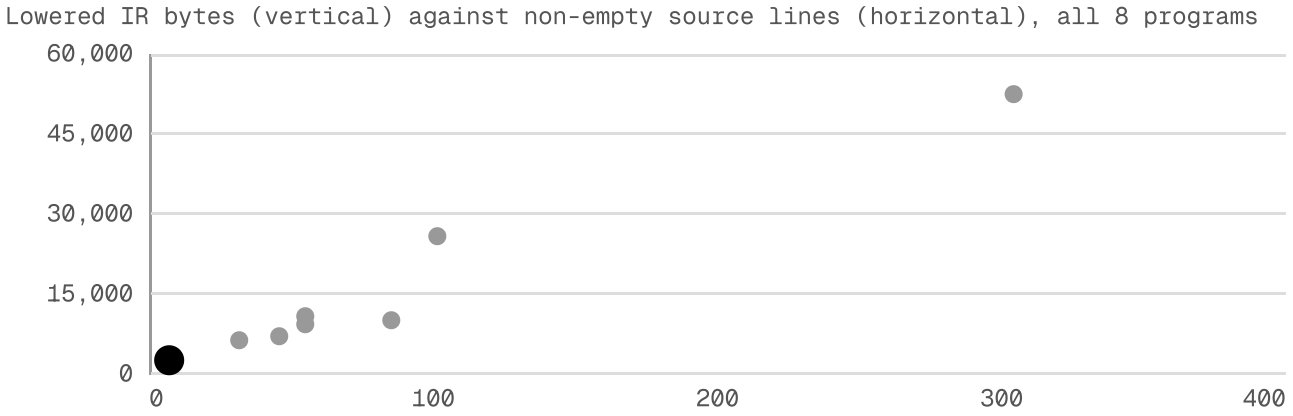
p01_hello · phases.coldTimed



7 of the 8 phases returned 0 ms; the bar for each is an empty track with a tick at the origin, not a missing row. Only `lower` registered any time at all.

Lowered IR size across the corpus

8 programs · backend.size.loweredIrBytes



Highlighted, drawn larger and ringed: p01_hello at 6 non-empty lines and 2,185 bytes of lowered IR.

Source size on the horizontal axis, the size of the MIR lowering produced on the vertical. The IR itself is never printed — `--emit llvm-ir` fails with BLD004 on every target — so its byte count is the only observable the compiler offers.

Every phase p01_hello reported, in the order the compiler lists them, from phases.coldTimed. Times are whole milliseconds as reported: a 0 means the phase returned inside one millisecond, not that it was skipped. Share is that phase as a percentage of the 4 ms total.

#	Phase	Reported	Share	Cacheable
1	resolve	0 ms	0%	yes
2	parse	0 ms	0%	yes
3	interface	0 ms	0%	yes
4	check	0 ms	0%	yes
5	lower	4 ms	100%	yes
6	codegen	0 ms	0%	yes
7	object	0 ms	0%	yes
8	link	0 ms	0%	no
All phases		**4 ms**	**100%**	—

The corpus ordered by non-empty source lines, smallest first, with the lowered IR size each program produced and the time its `lower` phase reported. Bytes per line is derived by division and is stated only to show the ratio is roughly stable. The selected program is marked in the first column.

Program	Non-empty lines	Lowered IR bytes	Bytes per line	lower
p01_hello · selected	6	2,185	364	4 ms
p02_arith	31	5,994	193	5 ms
p07_generics	45	6,573	146	5 ms
p03_control	54	10,340	191	6 ms
p05_errors	54	8,731	162	6 ms
p04_shapes	85	9,499	112	6 ms
p06_memory	101	25,416	252	12 ms
p08_lexer	304	52,110	171	17 ms

Read the two panels together: the front end costs nothing measurable because it reads facts already stored in `zero.graph` rather than re-deriving them from text, and `lower` is the only phase whose time, and whose output, grows with the size of the program.

Lowering cost by program — `zero time --json, cold`

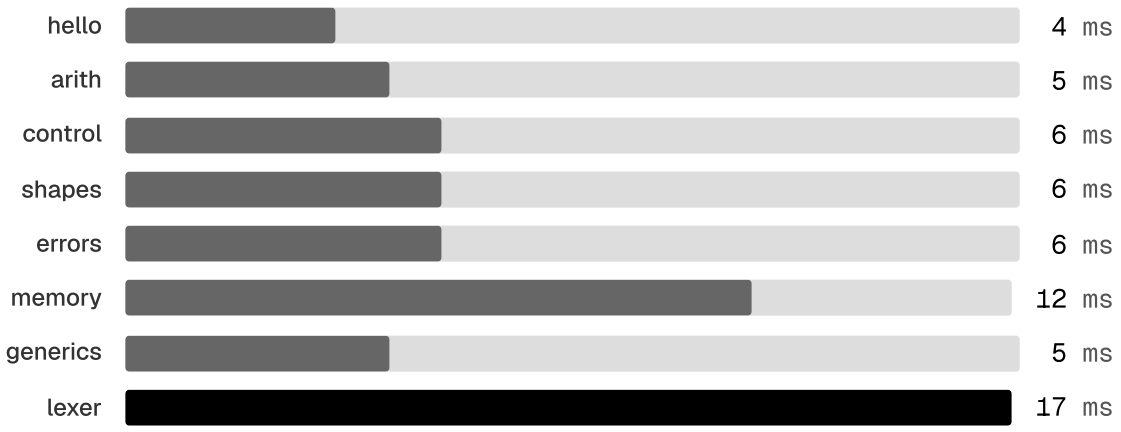


Figure 5. Milliseconds in the `lower` phase. Every other phase reports 0 ms for every program, so this is effectively the whole measurable pipeline.

Graph size against source size

8 programs

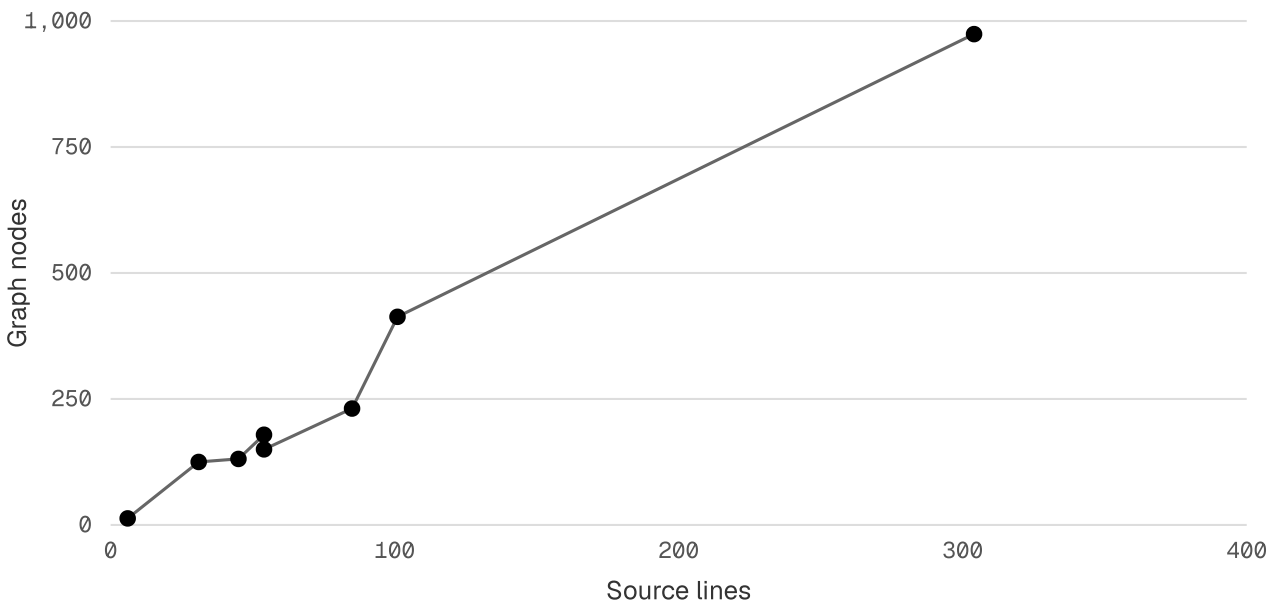


Figure 6. Graph nodes scale close to linearly with source lines (≈3.3 nodes per line). The graph is a constant-factor expansion of the program, not a blow-up.

This is the expected consequence of the architecture rather than an anomaly. In a text-first compiler the front end reconstructs meaning from characters on every invocation. In Zero it reads a store where names are already bound and types already recorded, so it does almost no work. What remains expensive is the one phase that cannot be cached away: translating graph facts into machine-level MIR.

The honest caveat is resolution. The compiler reports integer milliseconds, so "0 ms" means "under one millisecond", not "free". At our corpus scale the front end does not reach the reporting threshold. Resolving the front-end phases against each other would need a corpus perhaps two orders of magnitude larger, and we did not build one.

Phase composition, largest program

p08_lexer · 304 lines

p08_lexer, cold cache · total 17.0 ms

Figure 7. All eight reported phases for the tokenizer. The bar is effectively one segment: `lower` at 17 ms.

6.3 The symbol table, persisted

In the classical model the symbol table is a structure the front end builds and discards. In Zero it is sixteen persisted relations, carried between invocations in the graph store and reported as row counts on every compile.

All 8 corpus programs in corpus order, measured at each stage of the pipeline. Lines and bytes are source text; tok including newlines; nodes, edges, declarations, symbols and types are row counts in the persisted graph; typed no the host-target executable in bytes, or — where the backend refused to build it.

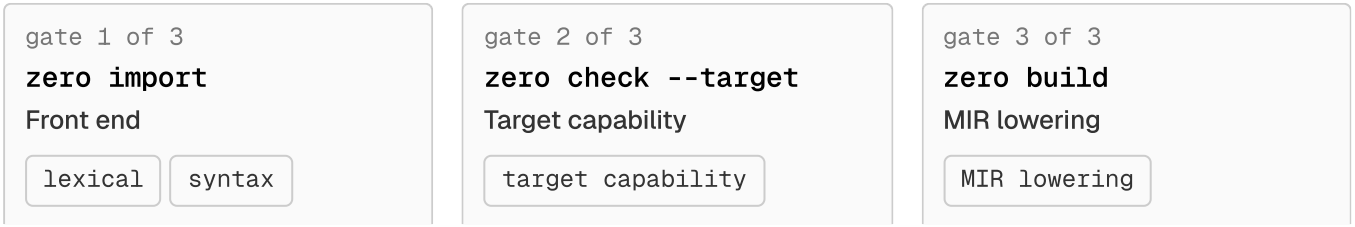
Program	Lines	Bytes	Tokens	Nodes	Edges	Decls	Symbols	Types
p01_hello	8	140	41	13	12	2	3	5
p02_arith	40	781	219	125	123	15	16	29
p03_control	62	1,624	297	179	177	16	17	31
p04_shapes	102	2,478	471	231	229	33	34	55
p05_errors	66	1,984	292	150	148	20	21	35
p06_memory	123	3,518	786	413	411	49	47	96
p07_generics	56	1,298	277	131	129	20	21	37
p08_lexer	335	10,390	1,804	974	972	108	109	176

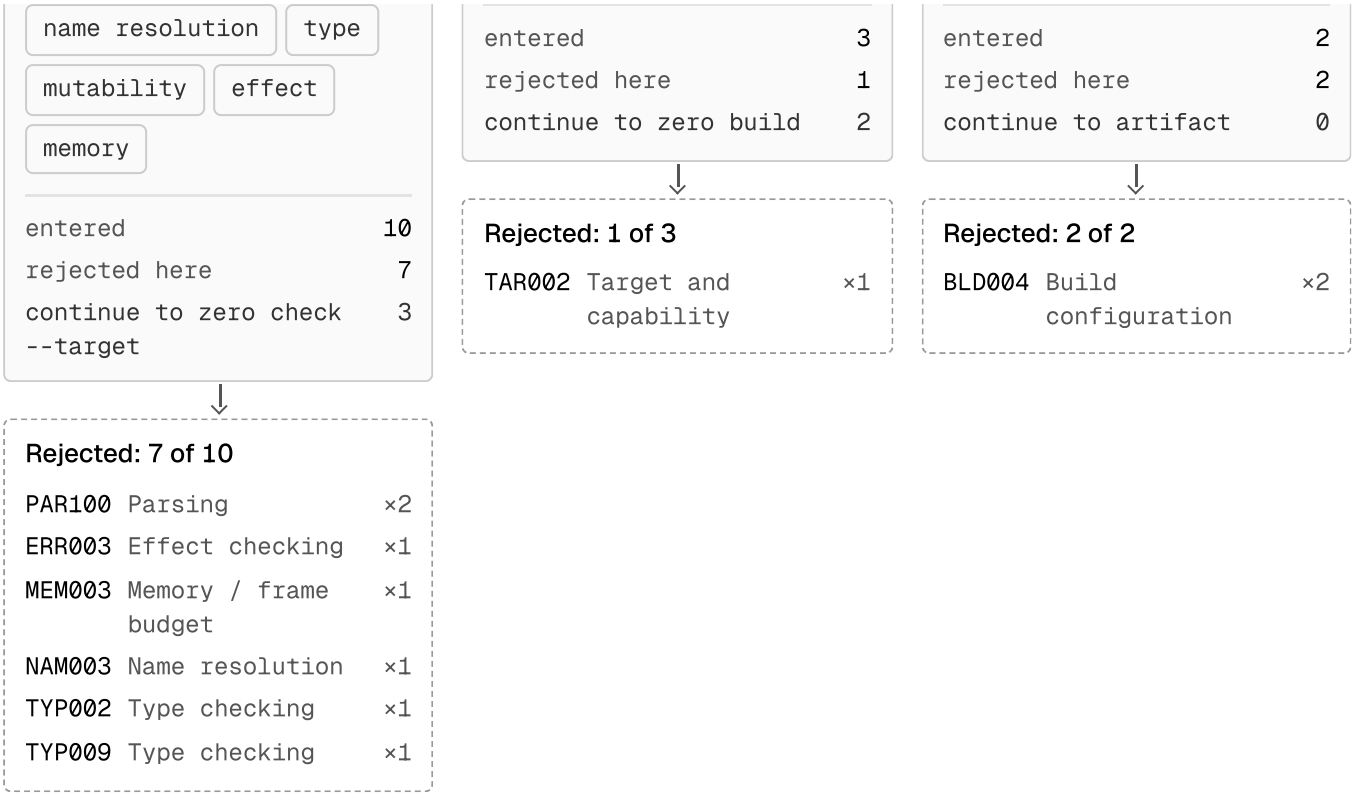
Two structural invariants hold across the entire corpus. The `sourceMap` row count equals the `node` count exactly for every program — every graph node retains a source position, which is what lets a diagnostic on a graph-derived fact still point at a line a human can read. And the edge count is exactly two fewer than the node count in every program, consistent with a spanning structure over the module and package roots.

6.4 Error handling: three admission gates, not one

This is the paper's central empirical result. We expected the error corpus to partition by phase within a single compile. Instead it partitions by *gate*, and there are three.

10 deliberately malformed programs enter at the left. Each gate admits what it cannot fault and turns the rest away downward.





Counts and codes are derived at render time from `capture.errorCases`: a case is attributed to the gate its `rejectedAt` field names, and the codes are the diagnostics that gate itself emitted. 0 of 10 malformed programs reach an artifact. On a narrow screen the gates stack, so the left-to-right flow becomes top to bottom; the rejected branch stays directly beneath its gate either way.

Text alternative to the diagram above: the three admission gates in order, with the population entering each, the number it rejected, the number it passed on, and the diagnostic codes it rejected them with. Population: the 10 error-corpus programs. Counts are programs, not diagnostic lines.

#	Gate	Checks	Entered	Rejected	Continued	Codes rejected with
1	`zero import`	lexical, syntax, name resolution, type, mutability, effect, memory	10	7	3	`PAR100` ×2, `ERR003` ×1, `MEM003` ×1, `NAM003` ×1, `TYP002` ×1, `TYP009` ×1
2	`zero check --target`	target capability	3	1	2	`TAR002` ×1
3	`zero build`	MIR lowering	2	2	0	`BLD004` ×2
Reached an artifact				—	0	—

The 10 deliberately malformed programs in the error corpus, one row each. The three gate columns record the outcome of `zero import`, `zero check` and `zero build`: “ok” passed, “rejected” was refused, — was never reached. No units.

Case	Targets phase	Import	Check	Build	Rejected at	Codes
`e01_lexical`	lexical	rejected	ok	—	import	PAR100
`e02_syntax`	syntax	rejected	ok	—	import	PAR100
`e03_name`	resolve	rejected	ok	—	import	NAM003
`e04_type`	check	rejected	ok	—	import	TYP002
`e05_mutability`	check	rejected	ok	—	import	TYP009
`e06_effect`	check	rejected	ok	—	import	ERR003
`e07_memory`	check	rejected	ok	—	import	MEM003
`e08_target`	target	ok	rejected	rejected	check	TAR002
`e09_lowering`	lower	ok	ok	rejected	build	BLD004
`e10_match`	—	ok	ok	rejected	build	BLD004

The 7 front-end failures — lexical, syntactic, name resolution, type, mutability, effect and memory — were all refused at `zero import`, the boundary at which text is admitted into the graph. Critically, `zero check` subsequently reported `ok` for all of them, because the malformed program never entered the store. We verified this directly: after each rejection, `zero view --fn main` still projected the *previous* program. In no case did an invalid program contaminate the graph.

The remaining cases separate cleanly. The capability violation passed ingestion and was refused by `zero check --target`, because whether a program is legal depends on which target you select — it is not a property of the program alone. And 2 cases passed both ingestion *and* checking and were refused only at `zero build`.

Work through the cases yourself below; each shows the real diagnostic the gate emitted, including its structured repair metadata.

Which gate stopped it, and what it said

Pick one of the 10 deliberately malformed programs. Zero offers three places to refuse it — admitting text into the graph, checking the stored graph, and building an artifact — and each panel below reports what that gate actually returned for this case.

Error case

e01_lexical

A byte the scanner cannot classify into any token kind.

`targets phase lexical · expected code PAR100 · target host default · rejected at import · codes PAR100`

Source as submitted

src/main.0 — 5 lines · 114 B

```
pub fn main(world: World) -> Void raises {
    let bad: i32 = 4 § 2
    check world.out.write("unreachable\n")
}
```

▶ Graph store after the attempt — storeContaminated: false

Three gates, in order

1 zero import — ■ rejected

Admits text into the graph. Scanning, parsing, name binding, type, effect and frame-budget checking all run here, before anything is written to zero.graph.

This is the gate that turned the program away.

`error · PAR100 · Parsing`

unexpected token in expression

Location	e01_lexical/src/main.0:2:22
Expected	expression
Actual	Â
Help	use canonical .0 text source
Fix safety	requires-human-review
Repair	repair-syntax — Repair the syntax at the reported parser span, then rerun zero check.

2 zero check — ○ passed

Re-checks the stored graph and reports target readiness alongside a top-level verdict.

Ran after the earlier rejection, so it judged the graph as it then stood, not the submitted text.

```
ok: true
targetReadiness.ok: true
targetReadiness.buildable: true
targetReadiness.stage: ready
```

No diagnostic recorded at this gate.

3 `zero build` □ not reached

Lowers the checked graph to MIR and emits an artifact for the selected target.

No diagnostic recorded at this gate.

The same three verdicts as a table, for the case shown above. Values are the raw `importOk`, `checkOk` and `buildOk` fields: — means the capture recorded null, which is the gate never being reached. No units.

Gate	Command	Field	Value	Outcome	Diagnostics
1	`zero import`	`importOk`	`false`	rejected — rejected here	1
2	`zero check`	`checkOk`	`true`	passed	0
3	`zero build`	`buildOk`	–	not reached	0

Each diagnostic above is the compiler's own record, not a rendering of its prose: a stable code, a span, an expected and an actual fact, a fix-safety rating and, where the compiler has one, a named repair. That is what makes a rejection something a program can act on rather than a paragraph a human must read.

6.5 The front end and the back end accept different languages

We built every corpus program for every advertised target: 64 combinations. 47 succeeded (73%). Every one of the 17 failures was BLD004 — the back end declining to lower a construct semantic analysis had already accepted.

Build outcome for every corpus program on every target the installed toolchain advertises (8 programs × 8 targets size in bytes; a cell holding a diagnostic code is a refusal, and the code is the marker.

Program	darwin-arm64	darwin-x64	linux-musl-x64	linux-musl-arm64
p01_hello	16,632	16,636	352	312
p02_arith	16,632	16,636	633	649
p03_control	16,632	16,636	795	867
p04_shapes	16,632	16,636	826	BLD004
p05_errors	16,632	BLD004	826	BLD004
p06_memory	16,632	16,636	2,439	BLD004
p07_generics	16,632	16,636	588	BLD004
p08_lexer	16,632	16,636	4,237	BLD004
Built	**8/8**	**7/8**	**8/8**	**3/8**

BLD004 · record — 9 builds

BLD004 · IR_VALUE_CHECK — 4 builds

BLD004 · unsupported instruction — 3 builds

BLD004 · IR_VALUE_RESCUE — 1 build

The failures are not random. They name specific constructs the MIR subset cannot represent: aggregate values crossing a function boundary (`record`), the `check` and `rescue` operators applied to user-defined fallible functions (`IR_VALUE_CHECK` , `IR_VALUE_RESCUE`), and instructions absent from a given architecture's emitter.

Two consequences follow. First, **backend completeness is target-specific, not language-specific.** The `p05_errors` program — which uses nothing but the documented `raise` / `check` / `rescue` forms — fails to build on the Windows host but cross-compiles to an 826-byte ELF for `linux-musl-x64` . The same graph, the same front end, two different answers, decided entirely by which emitter runs.

Second, **a passing `zero check` does not imply a buildable program** — though the reason is more specific than it first appears, and it took a second measurement to state correctly. The compiler is not ignorant. For our two lowering cases it sets `targetReadiness.buildable: false` , `stage: "lower"` , and attaches a BLD004 naming the exact construct. The analysis is performed and the answer recorded.

The difficulty is where it is recorded. In the same document the **top-level `ok` field reads `true` and the top-level `diagnostics` array is empty**; the command-line form prints `ok` . A consumer that tests `response.ok` , or checks whether `diagnostics` is empty — the two most natural things to do with this schema — concludes the program is fine.

Table 6. Fields reported by `zero check --json` for the two lowering cases. Both rows describe the same compilation; the middle columns and the right-hand columns disagree.

Case	Construct	`ok`	`diagnostics`	`readiness.buildable`	`readiness.stage`	Actual build
`e09_lowering`	`Outcome`	`true`	`0`	`false`	`lower`	`BLD004`
`e10_match`	`Match`	`true`	`0`	`false`	`lower`	`BLD004`

A third, quieter observation: the Mach-O emitter produces a 16,632-byte image for the six-line hello-world and the same 16,632 bytes for the 304-line tokenizer. Both are valid Mach-O arm64 executables. Artifact size on that target is page-quantised at 16 KiB and carries no information about program size in this range — a reminder that binary-size comparisons across object formats measure the format as much as the compiler.

The 8 targets the installed toolchain advertises, against the union of 9 capability names declared across them. “ye No units.

Target	OS	Arch	Object format	libc	args	env	fs
`darwin-arm64`	macos	aarch64	macho	default	yes	yes	yes
`darwin-x64`	macos	x86_64	macho	default	—	—	yes
`linux-musl-x64`	linux	x86_64	elf	musl	yes	yes	yes
`linux-musl-arm64`	linux	aarch64	elf	musl	—	—	—
`linux-x64`	linux	x86_64	elf	gnu	—	—	—
`linux-arm64`	linux	aarch64	elf	gnu	—	—	—
`win32-x64.exe`	windows	x86_64	coff	msvc	yes	yes	yes
`win32-arm64.exe`	windows	aarch64	coff	msvc	—	—	—

7. Interactive phase explorer

The artifact accompanying this paper lets a reader do what the classical diagram only depicts: take one program and watch it become each successive representation. Every panel shows real captured output from the command named in its caption — no reconstruction, no illustration.

One program through six phases

Pick a program and a phase. Each panel shows the artifact Zero 0.3.4 emitted for that program at that phase, together with the command that produced it.

Program

p01_hello — 8 lines

Line counts cover every file in the package; the source shown below is src/main.0.

1 Lexical analysis → 2 Syntax analysis → 3 Semantic analysis →
4 Intermediate code generation → 5 Code optimization → 6 Target code generation

src/main.0

```
pub fn main(world: World) -> Void raises {
    check world.out.write("hello from zero\n")
}
```

8	41	13	1,536 B
Source lines, all files	Tokens	Graph nodes	Artifact bytes

1 Lexical analysis

Character stream → Token stream. Carried by `parse`.

Token stream — src/main.0 · 26 tokens captured

pub fn main (world : World) -> Void raises {
check world . out . write ("hello from zero\n")
}

Token counts by kind. 41 tokens across 2 files, of which 35 are code tokens.

Kind	src/lib.0	src/main.0	Total
word	5	11	16
symbol	5	10	15
number	1	0	1
string	0	1	1
newline	3	3	6
eof	1	1	2
All kinds	**15**	**26**	**41**

Output of `zero tokens --json`: the character stream of `src/main.0` classified into 6 kinds, counted per file.

The interactive explorer is available in the web edition at zero.khe.money.

8. Discussion

8.1 Phases relocate; they do not disappear

It would be easy to read Zero's architecture as abolishing the front end. It does not. Every classical phase is present and every one still runs — but lexical, syntactic and semantic analysis have moved out of the compile path into an ingestion gate that runs once per edit rather than once per build. The steady-state compile path is phases four to six only.

This relocation explains all three of our findings at once: the timing distribution in §6.2 (a front end that reads instead of derives costs nothing), the containment result in §6.4 (a gate that refuses admission cannot propagate a malformed program downstream), and the acceptance gap in §6.5 (two gates maintained separately drift apart). One architectural choice, three consequences.

8.2 Two levels of truth in one document

Our first reading of the acceptance gap was wrong: we recorded it as the compiler failing to detect a problem. It detects it. `targetReadiness` carries the correct verdict, the correct stage, and a diagnostic naming the offending construct precisely enough to act on.

What the compiler does is subtler and, for its stated audience, arguably worse than not checking: it publishes two summaries of the same compilation that disagree. `ok: true` with an empty `diagnostics` array sits in the same JSON object as `buildable: false` with a BLD004. A human reading the whole document notices. A program reading the field the schema presents as the verdict does not.

This matters more here than elsewhere. A language whose central claim is that compiler output should be consumed by machines rather than parsed from English has taken on an obligation a human-facing compiler has not: its top-level fields are an API, and an agent has no independent means of doubting them. The fix is not more analysis — the analysis exists — but propagating its result into the fields that claim to summarise it. It is a useful reminder that machine-readability is necessary but not sufficient for machine-*reliability*: a schema also has to be internally consistent.

8.3 Diagnostics as data

Structured diagnostics are not new. What is different in Zero is the completeness of the commitment: there is no prose path carrying information the JSON path lacks, every code is stable, and each diagnostic carries a `fixSafety` rating and an optional typed `repair` identifier. For teaching error handling as a first-class phase concern, this is materially better raw material than a retrofitted JSON flag.

8.4 Implications for compiler pedagogy

Our practical conclusion is narrower than Zero's own marketing and, we think, more durable. Whether agent-oriented languages succeed depends mostly on pretraining distribution, and that argument is not ours to settle. But observability is valuable regardless of who wins it.

A student using Zero can print the phase list, time each phase, read the symbol table as sixteen relations with row counts, watch a program be refused at three distinct gates, and diff the object formats produced by five emitters from one source graph — all from the command line, without instrumenting a compiler or reading its source. We are not aware of another production-intent compiler where the phase structure is this directly inspectable.

---

9. Threats to validity

Single version, single platform. All measurements are from Zero 0.3.4 build 5b3a90a on one Windows x64 machine. Zero is explicitly experimental and shipped four minor versions in roughly a month; the backend gaps we document are the ones most likely to close. Cross-target builds were produced but only host and Linux artifacts were executed.

Corpus scale. 680 lines across 8 programs is small. It is adequate for demonstrating phase structure and the acceptance gap, both qualitative properties, but too small to resolve front-end phase timings against the compiler's 1 ms granularity or to claim asymptotic scaling. Our scaling claims are limited to the observation that graph size is a near-linear constant-factor expansion of source size over two orders of magnitude.

Corpus authorship. The corpus was written to exercise particular compiler tables, so coverage is deliberate rather than representative of programs people would actually write. Several programs were shaped by what the backend would accept, which biases them toward the lowerable subset — if anything this *understates* the acceptance gap.

Timing methodology. Wall-clock measurements include ~40 ms of process startup we could not separate without instrumenting the compiler binary. All phase-level claims therefore rest on the compiler's self-reported in-process times and inherit whatever measurement error it has.

Documentation discrepancies. Several examples in the shipped language guide do not compile on this build: the borrowing example raises BOR001, the errors example does not lower on the Windows emitter, and the shape-literal example is rejected by the test interpreter. We report these as observations about version 0.3.4, not as claims about the language design.

---

10. Related work

The phase decomposition we test against is due to Aho, Lam, Sethi and Ullman (https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472), with incremental and query-driven alternatives surveyed by Cooper and Torczon (https://www.elsevier.com/books/engineering-a-compiler/cooper/978-0-12-815412-0). Zero's persistent-graph approach has clear antecedents: the Roslyn (https://github.com/dotnet/roslyn) compiler platform exposed compilation as a queryable object model, and query-based architectures — rustc's demand-driven query system (https://rustc-dev-guide.rust-lang.org/query.html) and the salsa (https://github.com/salsa-rs/salsa) framework behind Rust Analyzer — already treat compilation as incremental recomputation over a memoised fact database rather than a fixed pipeline. Zero's distinguishing move is not the graph but making the graph the *authoritative stored artifact*, with text demoted to a projection.

On structured diagnostics, both rustc (https://doc.rust-lang.org/rustc/json.html) and TypeScript ship machine-readable error output; Zero's contribution is typed repair identifiers and safety ratings rather than machine-readability itself. The incremental re-analysis problem Zero solves with interface fingerprinting is the same one language servers solve with salsa-style memoisation.

We are not aware of prior empirical work measuring a compiler's phase structure through its own reporting interface, largely because few compilers expose one. The nearest analogues treat the compiler as a black box and measure only end-to-end time.

---

11. Conclusion

The classical six-phase model survives contact with a graph-first compiler, but not in the shape the diagram suggests. Across 8 programs and 64 build combinations we find that Zero implements every classical phase while relocating the first three out of the compile path into an ingestion gate. That single change explains our three measurements: lowering accounts for 100% of reported phase time because the front end reads rather than derives; all 7 front-end error cases were contained at the gate without contaminating the program store; and 17 of 64 builds failed on programs the front end had accepted, because the two gates maintain different notions of what the language is.

For a compilers course, the practical finding is that Zero makes the phases *visible* in a way mainstream toolchains do not. A student can time them, count the symbol table, and watch a program die at a specific gate — from the command line, without patching a compiler. Whether the language succeeds on its own agent-oriented terms is a separate question, and one whose answer probably has more to do with training-data distribution than with compiler design. The observability is worth studying either way.

References

A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools.* 2nd ed., Pearson, 2007. pearson.com

K. D. Cooper and L. Torczon. *Engineering a Compiler.* 3rd ed., Morgan Kaufmann, 2022. elsevier.com

Vercel Labs. *Zero — the programming language for agents.* zerolang.ai · source at github.com/vercel-labs/zerolang (Apache-2.0). Version 0.3.4, build 5b3a90a, studied here.

Vercel Labs. *Graph architecture.* zerolang.ai/concepts/graph-architecture — source of the agent-loop comparison reproduced in Figure 2.

Vercel Labs. *Compile path.* zerolang.ai/concepts/compile-path — source of the parse-first and graph-first pipelines reproduced in Figure 3.

Vercel Labs. *Semantic graph vs text.* zerolang.ai/concepts/semantic-vs-text

Vercel Labs. *Getting started.* zerolang.ai/getting-started. Version-matched language documentation was additionally retrieved from the installed compiler via `zero skills get language --full`.

Microsoft. *.NET Compiler Platform ("Roslyn").* github.com/dotnet/roslyn

N. Matsakis et al. *The rustc query system*, in the Rust Compiler Development Guide. rustc-dev-guide.rust-lang.org/query.html

The Rust Project. *JSON output (`--error-format=json`).* doc.rust-lang.org/rustc/json.html

salsa-rs. *salsa — a generic framework for on-demand, incrementalized computation.* github.com/salsa-rs/salsa

Vercel Labs. *wterm — a DOM-based terminal emulator with a WebAssembly core.* github.com/vercel-labs/wterm. Reviewed as a delivery option; see Appendix A.1.

Vercel. *Geist design system* and *Web Interface Guidelines*. vercel.com/geist · vercel.com/design/guidelines. Applied to the presentation of this paper, alongside WCAG 2.2 AA for contrast and structure.

Appendix A. Reproduction

The complete dataset behind this paper is one file, `web/data/capture.json`, regenerated by:

```
regenerate everything
```

```
curl -fsSL https://zerolang.ai/install.sh | bash
export PATH="$HOME/.zero/bin:$PATH"
zero --version          # 0.3.4 (build 5b3a90a)

npm run capture         # tools/capture.mjs -> web/data/capture.json
npm run qr              # tools/qr.mjs      -> web/components/qr-code.tsx
npm --prefix web run build
npm run pdf             # tools/pdf.mjs     -> paper/*.pdf
```

A.1 Why the explorer ships precomputed data

We considered compiling Zero programs in the browser, following wterm (https://github.com/vercel-labs/wterm), which runs a Zig-authored terminal core as WebAssembly. Zero 0.1.3 advertised `wasm32-wasi` and `wasm32-web` targets, which would have made this feasible. Version 0.3.4 advertises neither: the target list contains 8 native targets and no WebAssembly target, and the compiler's own `selfHostRouting` report marks `browserCompiler` as removed. In-browser compilation is not available on this build.

We also rejected running the compiler server-side. It would work — a Linux `zero` binary in a serverless function — but it would make a static document depend on a live execution environment, and every number would vary per request. Precomputing keeps the paper reproducible and lets the site deploy as pure static output.

A.2 Corpus contents

p01_hello, p02_arith, p03_control, p04_shapes, p05_errors, p06_memory, p07_generics, p08_lexer under `corpus/`, and e01_lexical, e02_syntax, e03_name, e04_type, e05_mutability, e06_effect, e07_memory, e08_target, e09_lowering, e10_match under `errors/`.

Harshit Khemani, Kush Ahuja, Mohit Kumar Mishra, Kushagra Agrawal · zero.khe.money

Zero 0.3.4 · 2026-08-09